ELIGITRACE: Evidence-Traced Clinical-Trial Eligibility Screening with a Fine-Tuned Small Language Model, KV-Cached and Speculative Decoding, and Tiled Online-Softmax Attention

Muhammad Ruhan Kamran
Department of Computer Science
FAST University (NUCES) Islamabad
Islamabad, Pakistan
Roll No. 23i-0062, Section CS-A

***Abstract*—Pre-screening patients for clinical trials means checking a free-text note against every eligibility criterion and citing the facts that justify each decision. We present ELIGITRACE, a locally executable system for criterion-level eligibility prediction with evidence tracing, built on Qwen2.5-1.5B-Instruct and the TrialGPT criterion-annotation benchmark. We fine-tune the model with LoRA on a 6 GB consumer GPU, choosing bf16 with gradient checkpointing after measuring the alternatives. On held-out test patients, the same model goes from 0.160 to 0.733 accuracy, from 0.155 to 0.528 macro-F1 over the labels present, and from 0.169 to 0.561 evidence micro-F1, with no unparseable outputs. We also implement and benchmark three inference mechanisms by hand:**

- **a KV-cached decoder, $3.6\times$ faster than recomputation at 128 context tokens and up to $20\times$ at 1,024;**
- **a tiled online-softmax attention kernel that matches a float64 reference to 2.5×10^{-6} in fp32 and uses $46\times$ less memory than unfused attention at 4,096 tokens;**
- **a greedy speculative decoder whose draft acceptance reaches 91% (96% with a fine-tuned draft), but which is still slower than plain cached decoding on launch-bound consumer hardware.**

Where bf16 outputs diverge from the reference, we trace the cause to logit ties within one bf16 rounding step and confirm exact agreement in fp32. A browser application screens unseen notes, highlights evidence sentences, explains each decision only from the cited evidence, and exposes the inference mechanisms live.

***Index Terms*—clinical-trial matching, LoRA, KV cache, speculative decoding, online softmax, evidence attribution**

I. INTRODUCTION AND PROBLEM STATEMENT

Clinical trials fail to recruit on time far more often than they fail scientifically, and a large part of the recruitment burden is *pre-screening*: a coordinator reads a free-text patient note against every inclusion and exclusion criterion of a protocol. The task is criterion-level, evidence-heavy and asymmetric—an absent fact is not a negative fact. Large proprietary models have shown strong results on this problem [1], [2], but they cannot be run locally on protected data, are expensive, and expose no control over inference.

We study the problem as defined by the TrialGPT criterion-annotation benchmark [1]: given (i) a patient note split into numbered sentences, (ii) a criterion type (inclusion or exclusion) and (iii) the criterion text, predict the expert eligibility label—one of four type-specific categories, including *not enough information* and *not applicable*—and the IDs of the patient sentences that support it. A deployable answer must also be *traceable*: each decision should point at its evidence and explain itself only from that evidence.

Contributions. We present ELIGITRACE, a complete, locally executable system built around Qwen2.5-1.5B-Instruct [3]:

1) a leakage-safe data pipeline and the prescribed patient-grouped 37/8/8 split with a published manifest and automated leakage tests;
2) a LoRA [4] adaptation whose precision strategy was *chosen by measurement* on a 6 GB consumer GPU, with a same-model before/after comparison on identical held-out examples;
3) a hand-written KV-cached greedy decoder, a tiled online-softmax attention kernel and a greedy speculative decoder, each validated against a reference (exact in fp32, and in bf16 up to logit ties within one rounding step) and benchmarked under controlled conditions;
4) a browser application that screens unseen notes against typed or fetched criteria, highlights the exact evidence sentences, aggregates criterion decisions with an explicit rule, exports a provenance-stamped report, and exposes a live inference laboratory.

Fig. 1 summarises the system.

II. RELATED WORK

Patient–trial matching. Koopman and Zuccon [5] released the patient topics that underlie the benchmark used here. TrialGPT [1] decomposes matching into criterion-level prediction with sentence-level evidence and shows that GPT-4 approaches expert accuracy; its criterion annotations (expert labels and expert evidence) form our mandatory dataset. Wornow et al. [2] reach similar conclusions zero-shot. Both rely on proprietary API models: excellent accuracy, but no local execution, no

control over decoding, and cost per criterion. We instead ask how far a 1.5 B-parameter open model adapted on 603 examples can go, and we keep the GPT-4 outputs shipped with the dataset strictly out of inputs, targets and evaluation.

Decoder architectures. The Transformer [6] remains the backbone. Modern small decoders combine rotary position embeddings (RoPE) [7], which make attention scores depend on relative position through rotations applied to queries and keys, and grouped-query attention (GQA) [8], which shares key/value heads across query heads and shrinks the KV cache. Qwen2.5 [3], Llama 3 [9] and Gemma 2 [10] all follow this recipe; they differ in licence, vocabulary and the availability of a smaller same-tokenizer sibling, which matters for speculative decoding (Section III-C).

Parameter-efficient adaptation. LoRA [4] learns a low-rank update $\Delta W = BA$ for frozen weights, cutting trainable parameters by orders of magnitude; QLoRA [11] additionally stores the frozen base in 4-bit NF4. Mixed precision [12] and gradient checkpointing [13] trade numerical range and recomputation for memory. These papers report memory and quality on data-centre GPUs; their trade-offs do not transfer directly to a 6 GB Turing card without tensor cores, which is why we measure them (Table III).

Efficient inference. Autoregressive decoding is memory-bandwidth bound; caching keys and values turns each step from $O(n)$ recomputation into a single-token pass [14], and PagedAttention [15] manages that cache at serving scale. Online softmax [16] lets the normaliser be computed in one streaming pass; Rabe and Staats [17] showed attention needs only $O(\sqrt{n})$ or $O(1)$ extra memory with chunking, and FlashAttention [18], [19] turned the same idea into fused, IO-aware GPU kernels. Speculative decoding [20], [21] drafts several tokens with a small model and verifies them in one pass of the large model, preserving the target distribution exactly (and, for greedy decoding, the exact token sequence). Its benefit depends on the draft/target cost ratio and the acceptance rate—both of which we measure rather than assume.

Limitations of prior work for our setting. API-based matchers cannot be benchmarked at the mechanism level; kernel papers report speed on hardware we do not have; and speculative-decoding gains are reported for large size ratios (e.g. 70 B targets) unlike our 1.5 B/0.5 B pair. ELIGITRACE connects these lines on one consumer GPU and reports where each technique helps and where it does not.

III. MATERIALS AND METHODS

A. Dataset and Leakage Control

We use only `ncbi/TrialGPT-Criterion-Annotations` (revision `1cfcafd`). It contains 1,015 criterion-level annotations for 53 patients and 103 trials. Notes are already split into numbered sentences (`"0. ...\n1. ..."`), so sentence IDs are taken verbatim and evidence is a set of those IDs. The columns `gpt4_explanation`, `gpt4_sentences`, `gpt4_eligibility`, `explanation_correctness` and TrialGPT's own `training` flag are dropped before any formatting. A test overwrites all of these columns with a sentinel string and asserts that every one of the 1,015 prompts and targets is byte-identical and sentinel-free, proving that model-generated outputs cannot reach the model. One annotation (ID 883) has an empty criterion text; it is kept, with an explicit marker, so that counts match the source. Labels are type-specific: inclusion criteria use {included, not included, not enough information, not applicable}; exclusion criteria use {excluded, not excluded, not enough information, not applicable}. The label distribution is highly skewed (e.g. only 10 *excluded*).

B. Patient-Grouped Split

Unique patient IDs are sorted lexicographically into a NumPy array, shuffled in place with `numpy.random.default_rng(42)`, and the first 37 / next 8 / last 8 patients form train / validation / test. Every annotation follows its patient. Table I gives the resulting counts; the manifest (`data/splits/split_manifest.json`) lists patient IDs and a SHA-256 hash of each split's annotation IDs. Automated tests check sizes, disjointness, full coverage, determinism and that every gold evidence ID exists in its note. Notably, the test split contains *no excluded* example, which affects how macro-F1 must be read (Section IV-A).

C. Model Selection

Table II compares five open-weight decoders. Facts are read from each model's `config.json`; parameter counts are exact (models instantiated on the `meta` device, tied embeddings counted once). We selected **Qwen2.5-1.5B-Instruct**: Apache-2.0 and ungated; RoPE + GQA with only 2 KV heads, giving the smallest KV cache of the candidates (28 KiB/token in 16-bit); a 32 K context far above our longest example (623 tokens); 1.54 B parameters (2.9 GiB in bf16) that leave room on a 6 GB GPU for activations, the KV cache *and* a draft model; and, decisively for Task 5, an official **Qwen2.5-0.5B-Instruct** sibling with an identical tokenizer (151,665 entries, identical chat template, identical 151,936-wide output layer). Llama-3.2-1B has no smaller same-tokenizer sibling and a gated community licence; Gemma-2-2B (2.6 B) and Qwen3-1.7B (1.72 B, 8 KV heads $\Rightarrow 4\times$ larger cache) fit less comfortably next to a draft model.

D. Input/Output Format and Tokenisation

One prompt template (Qwen chat format) is shared by fine-tuning, evaluation and the application. The system message instructs the model to judge one criterion from the numbered note only, to answer *not enough information* when facts are missing ("missing evidence is never proof"), and to reply with JSON. The user message contains the numbered note, the criterion type, the criterion text and the four allowed labels for that type. The target is a compact JSON object, e.g. {`"label": "included", "evidence": [0, 3]`}. With Qwen's byte-level BPE tokenizer the longest training example is 623 tokens (median 311; Fig. 2); we set

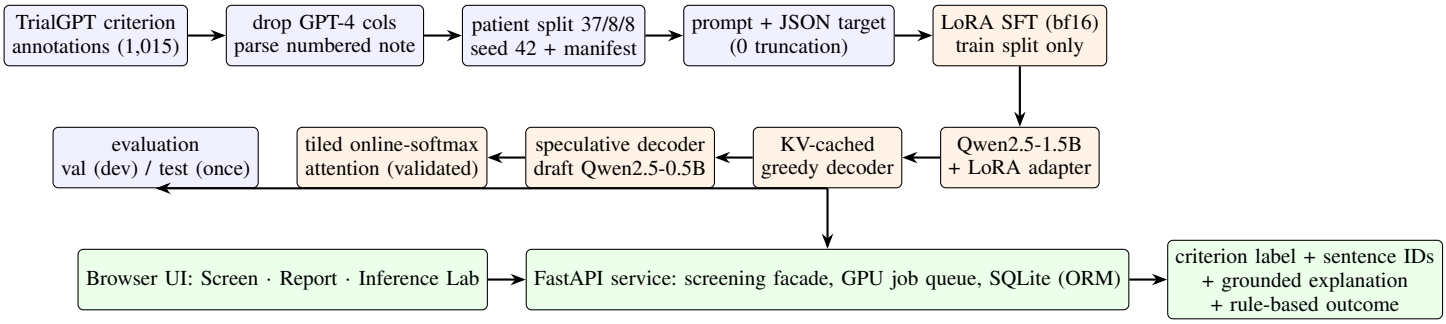


Fig. 1. ELIGITRACE architecture. The offline pipeline (top) prepares leakage-safe data and adapts the model; the inference core (middle) contains the three hand-written mechanisms; the application (bottom) exposes screening and live benchmarking in the browser.

TABLE I
PATIENT-GROUPED SPLIT (SEED 42) AND EXPERT LABEL COUNTS.

Split	Pat.	Ann.	Inc.	Exc.	incl	not incl	excl	not excl	NEI	N/A
train	37	603	215	388	88	13	8	292	174	28
val	8	206	66	140	35	3	2	101	53	12
test	8	206	75	131	27	7	0	91	66	15

TABLE II
CANDIDATE DECODERS (FROM CONFIG.JSON; EXACT PARAMETER COUNTS; 16-BIT MEMORY). KV = KIB OF CACHE PER TOKEN. ALL USE ROPE, RMSNORM/LAYERNORM VARIANTS AND GQA.

Model	Params (B)	L	Hq/Hkv	d	Vocab	Ctx	GiB	KV	Licence
Qwen2.5-1.5B-Instruct	1.54	28	12/2	128	151k	32k	2.9	28	Apache-2.0
Qwen2.5-0.5B-Instruct	0.49	24	14/2	64	151k	32k	0.9	12	Apache-2.0
Qwen3-1.7B	1.72	28	16/8	128	151k	40k	3.2	112	Apache-2.0
Llama-3.2-1B-Instruct	1.24	16	32/8	64	128k	128k	2.3	32	Llama 3.2 CL
gemma-2-2b-it	2.61	26	8/4	256	256k	8k	4.9	104	Gemma ToU

`max_seq_len`=1024, so **no example is truncated and no evidence sentence is lost**—verified by an audit that would list any truncated annotation and any gold sentence beyond the cut. A robust parser extracts the first JSON object, normalises label synonyms, rejects labels not allowed for the criterion type (counted as *invalid*, never silently mapped), and drops cited IDs that do not exist in the note (reported as unsupported citations).

E. Fine-Tuning

Objective. Supervised fine-tuning with the causal LM cross-entropy computed only on assistant (target) tokens; all prompt tokens are masked with -100, so the model learns to emit the label and evidence, not to reproduce notes. The `<|im_end|>` token is part of the target, teaching the model to stop.

Precision strategy chosen by measurement. Our GTX 1660 SUPER (6 GB, Turing `sm_75`, no tensor cores) behaves unlike the GPUs in the literature. A micro-benchmark gave 3.9 TFLOP/s for fp32 GEMM, 2.1 for bf16 but only 0.43 for fp16 (cuBLAS fp16 with fp32 accumulation is slow on TU116). We therefore ran every candidate training configuration for four optimizer steps on the *longest* training example in a fresh process (Table III). The Windows driver's "CUDA sysmem fallback" initially let fp32 silently spill into system RAM (a 100× slowdown instead of an error); capping the allocator at the physical VRAM made out-of-memory explicit. Full fp32 does not fit (6.2 GiB of weights); accelerate's CPU offload is incompatible with PEFT gradients; fp16 autocast works but is 3.6× slower; QLoRA saves 0.9 GB but quantises the frozen weights. **bf16 storage of the frozen base + fp32**

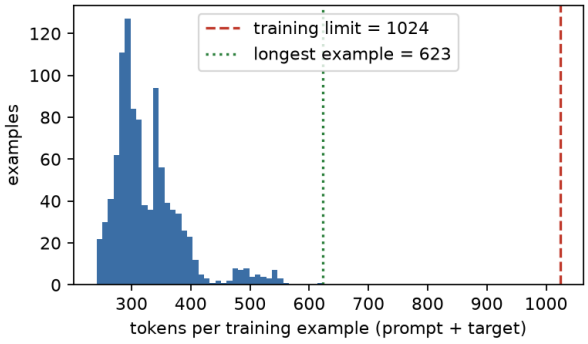


Fig. 2. Tokens per training example (prompt + target) across all 1,015 annotations. The red dashed line is the training limit (1,024 tokens); the green dotted line marks the longest example (623 tokens). Every example fits, so truncation never removes evidence. A looser 2,048-token audit also finds no example over the limit (`results/data/token_stats.json`).

TABLE III
MEASURED TRAINING CONFIGURATIONS ON NVIDIA GEFORCE GTX 1660 SUPER (6.0 GB): LORA R=16, LONGEST TRAINING EXAMPLE (623 TOKENS), 4 OPTIMIZER STEPS, FIRST STEP EXCLUDED FROM TIMING.

Strategy	Ckpt	MB	Weights GiB	Peak GiB	s/step	Status
fp32	yes	1	–	–	–	OOM
fp32-offload	yes	1	–	–	–	incompatible
fp16-mixed	yes	1	2.94	4.22	13.80	ok
bf16-mixed	no	1	–	–	–	OOM
bf16-mixed	yes	1	2.94	4.22	3.87	ok
bf16-mixed	yes	4	–	–	–	OOM
bf16-mixed	yes	8	–	–	–	OOM
qlora-nf4	yes	1	1.58	3.33	3.71	ok
qlora-nf4	yes	4	–	–	–	OOM

TABLE IV
FINE-TUNING CONFIGURATION.

Setting	Value
Base model	Qwen/Qwen2.5-1.5B-Instruct
Precision	bf16 base, fp32 LoRA, bf16 autocast
LoRA r / α / dropout	16 / 32 / 0.05
Target modules	q,k,v,o,gate,up,down
Optimizer	AdamW, wd 0.0
LR / schedule	0.0002 / cosine, 5% warm-up
Batch	1×16 accumulation
Epochs / clip / seed	3 / 1.0 / 42
Max length	1024 (no truncation)
Grad. checkpointing	True
Selection	best val macro-F1 over epochs

LoRA weights + bf16 autocast + gradient checkpointing is the fastest configuration that keeps the original weights; any micro-batch above one exceeds memory because the 151,936-way output logits dominate activations, so we use micro-batch 1 with 16 accumulation steps.

Hyper-parameters (Table IV): LoRA rank 16, α=32, dropout 0.05 on all seven attention and MLP projections (18.5 M trainable parameters, 1.2% of the model), AdamW [22] with learning rate 2×10^{-4}, cosine schedule with 5% warm-up, gradient clipping at 1.0, 3 epochs, seed 42. After each epoch the adapter is saved and the *validation* split is decoded with the exact evaluation decoder; the epoch with the best validation macro-F1 (ties: evidence micro-F1) is selected. The test split is never used for selection.

F. Grounded Explanations

The dataset's only free-text explanations are GPT-4 outputs, which we refuse to use as targets (leakage, and 124 of them are marked partially correct or incorrect). The fine-tuned model therefore emits only label and evidence, and the explanation is produced afterwards in one of two ways. (i) *Grounded LLM*: the same network with the LoRA adapter *disabled* (i.e. the instruction-tuned base) receives only the criterion, the decided label and the text of the cited sentences, and writes one or two sentences. A deterministic grounding check rejects any rationale that mentions a number absent from the cited evidence or the criterion, or that contradicts the label. (ii) *Template*: a deterministic sentence quoting the cited evidence verbatim. Rejected LLM rationales fall back to the template, so every explanation shown is grounded in the cited sentences.

Algorithm 1 KV-cached greedy decoding (batch 1)

$C \leftarrow$ empty cache; $\ell \leftarrow f(x_{0:T};\ \mathrm{pos} = 0..T{-}1, C)$ ▷ prefill, appends T K/V
$y_1 \leftarrow \arg\max \ell_{T-1}$
for $t = 1$ **to** $N - 1$ **while** $y_t \notin \mathrm{EOS}$ **do**
  $\ell \leftarrow f(y_t;\ \mathrm{pos} = T{+}t{-}1, C)$ ▷ one token; cache grows by 1
  $y_{t+1} \leftarrow \arg\max \ell$
end for
assert $|C| = T + |y| - 1$

G. KV-Cached Autoregressive Decoding

We implement greedy decoding directly on model forward passes (Alg. 1). In prefill the full prompt of length T is processed with `position_ids` $= 0..T-1$; each layer stores keys (already rotated by RoPE) and values in a `DynamicCache` of shape $[B, H_{kv}, T, d]$. Each decode step feeds only the newest token with absolute position $T + t$ (`position_ids` and `cache_position`) and an attention mask extended by one; the cache grows by one entry per layer. Because RoPE is applied before caching, cached keys retain their absolute positions; a wrong position for the new query silently changes the output, which our equality tests would detect. A causal mask is needed in prefill but not for a single query at the last position. The reference path re-runs the whole sequence without cache at every step. Assertions check the cache length after prefill and after decoding, and a trace records the layer-0 key shape at every step (Section V-F). With GQA, the cache stores $H_{kv} = 2$ heads instead of $H_q = 12$: $2 \cdot L \cdot H_{kv} \cdot d \cdot 2\,\mathrm{B} = 28\,\mathrm{KiB}$ per token for L=28, d=128.

H. Tiled Attention with Online Softmax

The reference computes $S = QK^\top/\sqrt{d}$ in fp32, masks with $-\infty$ (causal convention: query i at absolute position $S_k{-}S_q{+}i$ may see keys $j \le S_k - S_q + i$, which also covers single-token decode), subtracts the row maximum before exponentiation, normalises, and multiplies by V; fully masked rows return zeros. The tiled kernel (Alg. 2) never forms S: for each block of B_q queries it streams blocks of B_k keys, keeping a running maximum m, normaliser ℓ and un-normalised output A. When a new block raises the maximum, previous statistics are rescaled by $\alpha = e^{m-m'}$ [16], [18]. Key blocks entirely above the diagonal are skipped; only diagonal blocks are masked. GQA is handled by repeating KV heads. All accumulation is in fp32.

I. Greedy Speculative Decoding

Draft D (Qwen2.5-0.5B-Instruct, 0.49 B) and target M (fine-tuned 1.5 B) keep separate KV caches. With s the current sequence of length L, the invariant is that M's cache holds $s_{0:L-1}$ and D's holds $s_{0:L-1}$ or $s_{0:L-2}$; the last token is pending. Each round (Alg. 3): D ingests its pending token(s)

Algorithm 2 Tiled attention with online softmax (one query block)

$m \leftarrow -\infty,\ \ell \leftarrow 0,\ A \leftarrow 0$ ▷ per query row
for each key block j visible under the causal mask **do**
  $S_j \leftarrow Q_i K_j^\top / \sqrt{d}$; mask inside the diagonal block
  $m' \leftarrow \max(m, \mathrm{rowmax}(S_j));\ P \leftarrow e^{S_j - m'};\ \alpha \leftarrow e^{m-m'}$
  $\ell \leftarrow \alpha\ell + \mathrm{rowsum}(P);\ A \leftarrow \alpha A + PV_j;\ m \leftarrow m'$
end for
$O_i \leftarrow A/\ell; \quad \mathrm{LSE}_i \leftarrow m + \log \ell$

Algorithm 3 Greedy speculative decoding: one round

D: ingest pending token(s); propose $d_{1..\gamma}$ greedily
M: one forward on $[s_{L-1}, d_{1..\gamma}] \Rightarrow g_{1..\gamma+1}$
$n \leftarrow \max\{k : d_{1..k} = g_{1..k}\};\ \ s \leftarrow s \,\|\, d_{1..n} \,\|\, g_{n+1}$
M.crop($|s| - 1$); D.crop($\min(|D|, |s| - 1)$) ▷ drop rejected K/V
stats: proposed $+\!=\!\gamma$, accepted $+\!=\!n$

and greedily proposes γ tokens; M runs *one* forward over $[s_{L-1}, d_1..d_\gamma]$ at positions $L{-}1..L{-}1{+}\gamma$, yielding its own greedy choices $g_1..g_{\gamma+1}$; the longest prefix with $d_i = g_i$ is accepted and g_{n+1} is appended (a correction if $n < \gamma$, a free bonus token if $n = \gamma$). Both caches are then cropped (`DynamicCache.crop`) so that no key/value of a rejected token survives. EOS or the token budget inside an accepted block truncates exactly. Every emitted token is the target's own greedy choice for the same prefix, so the output must equal plain greedy decoding; cache-length assertions run after every round.

J. Application Workflow

The FastAPI service follows a router → service → repository layering with Pydantic request schemas (constraints double as input validation: note 20–12,000 characters, 1–40 criteria, typed labels, NCT-ID pattern, $\gamma \in [1, 8]$). A singleton model registry loads the target (with the adapter kept *unmerged* so explanations can disable it) and the draft once, in the background; a single-worker executor serialises GPU jobs. A screening is created as a job whose criteria are decoded one by one and persisted (SQLite through SQLAlchemy), while the browser polls progress. Free-text notes are segmented into numbered sentences; criteria can be pasted (split on Inclusion/Exclusion headings), typed, or fetched by NCT ID from the public ClinicalTrials.gov API. The overall preliminary outcome is an explicit rule, never a predicted label: any inclusion *not included* or exclusion *excluded* ⇒ *Likely ineligible*; otherwise any *not enough information* (or unparseable output) ⇒ *Needs more information*; otherwise *Likely eligible*; *not applicable* is neutral. Reports are exported as printable HTML, Markdown or JSON with provenance (model, adapter SHA-256, precision, decoding settings, commit, timestamp) and the disclaimer. The Inference Lab runs no-cache, KV-cached and speculative decoding live on a validation prompt or a custom prompt, and the reference/tiled/PyTorch attention kernels against a float64 ground truth.

IV. Experimental Setup and Metrics

Hardware and software. All training, evaluation and benchmarking ran locally on one NVIDIA GeForce GTX 1660 SUPER (6 GB GDDR6, Turing `sm_75`, no tensor cores), an AMD Ryzen 5 2600 and 16 GB RAM, under Windows 11 with driver 621.14, Python 3.12, PyTorch 2.14.1 (CUDA 12.6), Transformers 4.57.6 and PEFT 0.21.2 (exact versions pinned in `uv.lock`). No paid service or cloud GPU was used. The application also runs on CPU (slowly) and on the author's second machine (RTX 4050, 6 GB).

Decoding settings (all comparisons). Greedy argmax decoding (no sampling, no repetition penalty), batch size 1, at most 64 new tokens, bf16 weights, the project's own KV-cached decoder, and stopping at `<|im_end|>`. Base and fine-tuned models use the identical prompt, parser and examples. Benchmarks disable EOS so that every run produces exactly the requested number of tokens.

Protocol. Model and epoch selection used only the validation split (8 patients, 206 annotations). The test split (8 patients, 206 annotations) was decoded exactly once by `evaluation/final.py`, which first writes a `FROZEN.json` record (adapter SHA-256, commit, prompt hash, decoding settings) and refuses to run again; ordinary evaluation commands raise an error when asked for the test split.

A. Metrics

Criterion classification. Accuracy; macro-F1 over all six labels (a class absent from the gold labels and never predicted scores $F_1 = 0$); macro-F1$_p$ over the labels present in the gold set of that split (needed because the test split has no *excluded* example); per-class precision, recall and F1; and confusion matrices. Everything is reported overall and separately for inclusion criteria (4 labels) and exclusion criteria (4 labels). Unparseable outputs or labels invalid for the criterion type are a separate *invalid* prediction that always counts as an error. Because the 8 test patients are the independent units, we report 95% patient-level bootstrap intervals (1,000 resamples).

Evidence localisation. For each record with gold set G and predicted set P of sentence IDs: $P{=}G{=}\emptyset$ gives precision = recall = F1 = 1 (correct abstention); $G{=}\emptyset \neq P$ gives precision 0, recall 1, F1 0 (hallucinated evidence); $G \neq \emptyset{=}P$ gives precision 1, recall 0, F1 0 (missed evidence); otherwise standard set precision/recall/F1. *Micro* scores pool TP/FP/FN over all records (empty–empty records contribute nothing); *macro* scores average the per-record values. Because conventions could inflate scores, we also report micro-F1 on the subset with non-empty gold evidence, and count correct abstentions, hallucinations and cited IDs that do not exist in the note.

Inference benchmarks. Same model and inputs for every compared path; warm-up runs first (one for decoding, two for attention), then repeated timed runs (three for KV-cache, two per prompt for speculative decoding, five

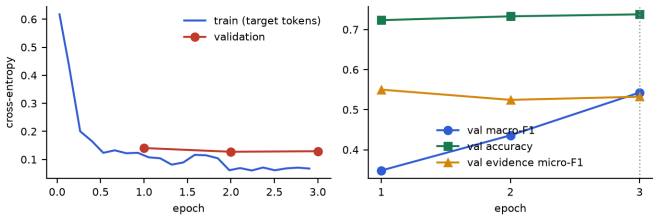


Fig. 3. Training loss per optimiser step, and validation loss and generated-output validation macro-F1 per epoch.

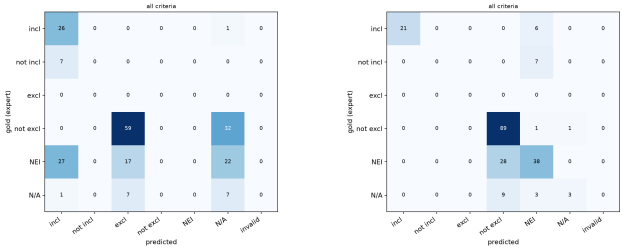


Fig. 4. Test confusion matrices over all six labels: zero-shot base (left) and fine-tuned (right).

for attention) with `torch.cuda.synchronize()` around each timed region; peak memory is `torch.cuda.max_memory_allocated()` after `reset_peak_memory_stats()`, reported as extra memory over the resident weights. KV-cache inputs of 128/512/1024 tokens are built from real validation prompts (several notes concatenated for the longest), with 16/64/128 output tokens (the quadratic no-cache reference makes longer outputs impractical: 128 tokens after a 1,024-token input already take about four minutes per run).

V. Results and Analysis

A. Training

Training took 62 min for 3 epochs (114 optimiser steps, 3.75 ks wall clock) with a peak of 4.28 GB of allocated VRAM. Fig. 3 shows the curves. Validation loss was lowest after epoch 2 (0.127 vs. 0.129 after epoch 3), but generated-output validation macro-F1 kept rising (0.348, 0.436, 0.543). We selected the checkpoint by validation macro-F1, the metric we report, so epoch 3 was used. Token-level loss is dominated by the evidence-ID tokens and the JSON scaffold, so it is a weak proxy for the label decision.

B. Classification: Before vs. After Fine-Tuning

Table V compares the same network with and without the adapter on identical inputs, using the same prompt, decoder and parser. On the held-out test patients, test accuracy rose from 0.160 to 0.733 (patient-level bootstrap 95% CI [0.583, 0.843]). Macro-F1 over the labels present in gold rose from 0.155 to 0.528 (CI [0.436, 0.608]). Neither model produced an unparseable output, so the gain comes from label and evidence decisions, not from formatting. The zero-shot base mostly answers *included* or *not applicable* and almost never *not enough information*: it treats absent facts as decisive.

Six-label macro-F1 on test (0.440) is lower than on validation (0.543) mainly because the test split contains no *excluded* example, which scores that class as 0. Over the labels present in gold the two splits agree (0.528 vs. 0.543).

Per class (Table VI), the model is strong and precise on *included* (P = 1.00, F1 0.875) and *not excluded* (F1 0.820). It is weak on the rare classes:

- *not included*: all 7 test cases were predicted *not enough information*;
- *not applicable*: recall is 0.20.

These are the classes with the fewest training examples, so the result matches the label skew documented in Section III-B. For exclusion criteria, 28 of the 66 *not enough information* cases were predicted *not excluded*. This reproduces, in a milder form, the base model's habit of reading silence as a negative finding. The confusion matrices in Fig. 4 show the same pattern.

C. Evidence Retrieval

Fine-tuning improves evidence citation more than any other metric (Table VII). Micro-F1 rose from 0.169 to 0.561, and the gain came almost entirely from precision (0.097 to 0.500), with recall unchanged (0.649 vs. 0.638). The base model cites many sentences indiscriminately. The fine-tuned model abstains correctly on 131 of the 145 empty-gold criteria, compared with 53 for the base model. On the non-empty-gold subset, which isolates retrieval from abstention, F1 rose from 0.374 to 0.663. Neither model cited a sentence ID outside the note.

D. Error Analysis

Of the 206 test criteria, 55 received the wrong label. Table VIII groups them with deterministic rules (`evaluation/errors.py`). The most frequent pattern is an unsupported decision, where missing information is treated as decisive (28 cases). Temporal qualifiers such as "within the past 6 months" and "history of" are next (19 cases). The opposite error, answering *not enough information* although the experts found the note sufficient, occurs 17 times.

One instructive case is a vascular-surgery trial whose inclusion criteria list specific operations ("patients undergoing open abdominal aortic aneurysm repair"). For a patient with a femoral pseudoaneurysm, the experts labelled these criteria *not included* with no evidence: a note that describes a different procedure implies the listed one is not happening. The model answered *not enough information* and cited the three sentences describing the actual procedure. Expert reasoning of this kind rests on the absence of a fact in an otherwise complete clinical picture, and it is underrepresented in 603 training examples.

A further 34 records have the correct label but differ from the gold evidence. Most of these are omissions on *not excluded* criteria, where the annotators cited background sentences such as the demographic opening line.

TABLE V

SAME MODEL BEFORE AND AFTER FINE-TUNING ON IDENTICAL EXAMPLES (GREEDY, KV-CACHED DECODER, BF16, MAX 64 NEW TOKENS). MACRO-F1 OVER ALL 6 LABELS; $MF1_p$ OVER LABELS PRESENT IN GOLD; INC./EXC. = 4-LABEL MACRO-F1 PER CRITERION TYPE; EV = EVIDENCE F1; INV = % UNPARSEABLE OUTPUTS.

Model / split	Acc	MF1	$MF1_p$	Inc.	Exc.	Ev_μ	Ev_M	Inv
Base (zero-shot), val	0.189	0.145	0.145	0.243	0.030	0.175	0.304	0.0
Fine-tuned, val	0.738	0.543	0.543	0.383	0.520	0.532	0.731	0.0
Base (zero-shot), test	0.160	0.129	0.155	0.207	0.042	0.169	0.354	0.0
Fine-tuned, test	0.733	0.440	0.528	0.425	0.299	0.561	0.819	0.0

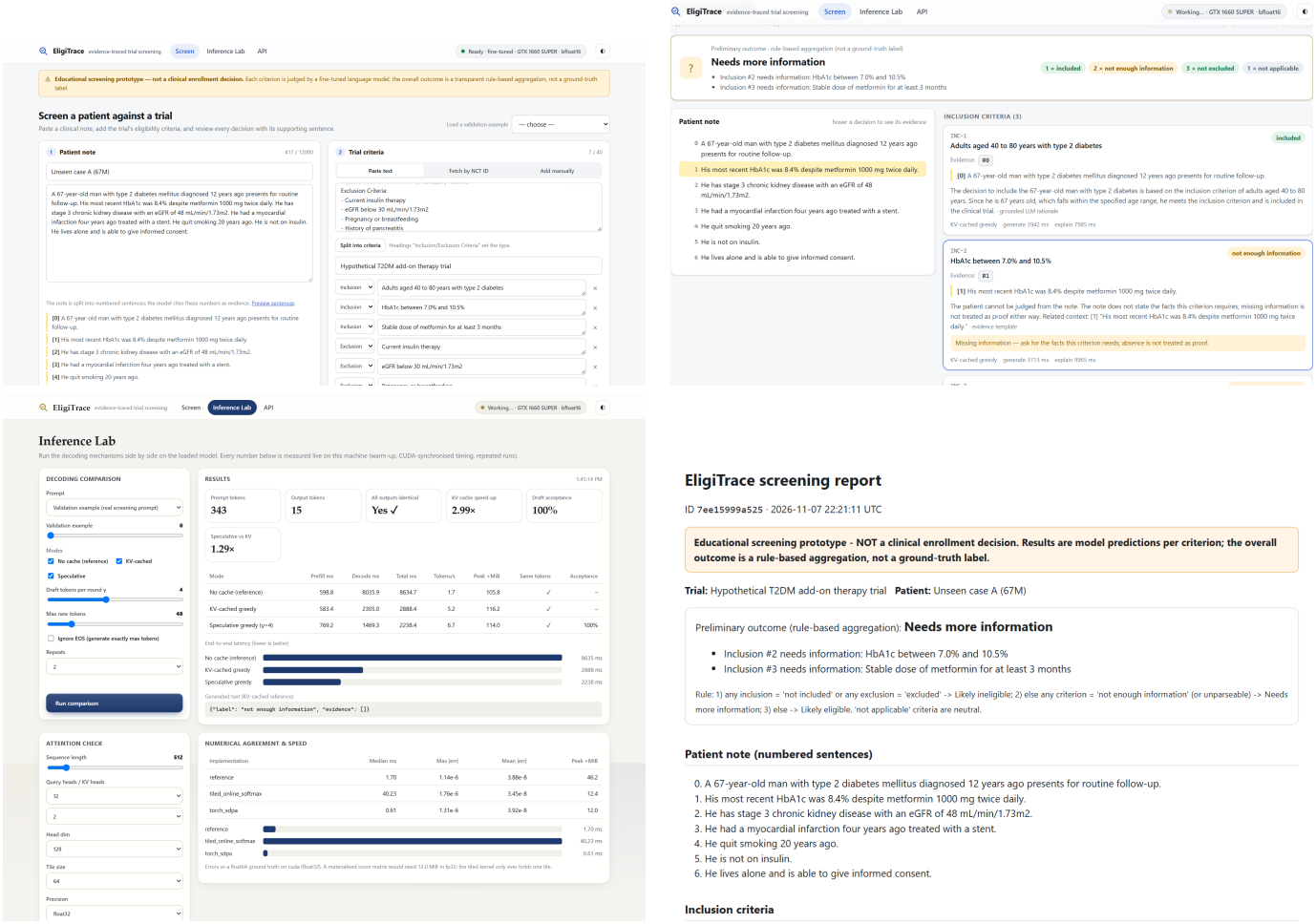


Fig. 5. ELIGITRACE application. Top: input of an unseen note and criteria (left), and criterion results with highlighted evidence, grounded rationales and the rule-based outcome (right). Bottom: Inference Lab with live decoder and attention comparisons (left), and the exported report with provenance (right).

E. Seeds, Oversampling, QLoRA and Rank (Validation Only)

The main run uses one seed and one configuration, so we measured how much of a result is configuration and how much is chance. Every run below repeats the pipeline of Section III-E on the *training* split with one change, is scored on the 206 *validation* criteria with the same prompt, parser and greedy decoder, and never touches the test split, which was evaluated once (Table V). Seeds, QLoRA, rank and learning-rate runs are scored at the final epoch only (epoch 3, which was also the baseline's best epoch) to fit the time budget.

Seeds. Over seeds 42, 43 and 44 the validation accuracy is 0.749 ± 0.010 (range 0.738–0.757), so accuracy is stable to about one point. Macro-F1 is not: 0.534 ± 0.079 (range 0.450–0.608). The reason is the label counts. The validation split has 2 *excluded* and 3 *not included* criteria, and the F1 of those classes jumps between 0 and 1 from one seed to the next, which moves the six-class average by up to 0.15. Seed 42, the model used for the frozen test evaluation, has a validation macro-F1 of 0.543, close to the three-seed mean, so the main result is typical rather than a favourable draw. Differences below roughly 0.15 macro-F1 between single runs should not be read as real.

TABLE VI
PER-CLASS RESULTS ON THE HELD-OUT TEST SPLIT (FINE-TUNED; BASE F1 FOR REFERENCE). CLASSES WITH ZERO SUPPORT CANNOT BE SCORED.

Type	Label	n	$F1_{base}$	P	R	F1
inc.	included	27	0.591	1.000	0.778	0.875
inc.	not included	7	0.000	0.000	0.000	0.000
inc.	not enough information	38	0.000	0.704	1.000	0.826
inc.	not applicable	3	0.235	0.000	0.000	0.000
exc.	excluded	0	0.000	0.000	0.000	0.000
exc.	not excluded	91	0.000	0.706	0.978	0.820
exc.	not enough information	28	0.000	0.000	0.000	0.000
exc.	not applicable	12	0.167	0.750	0.250	0.375

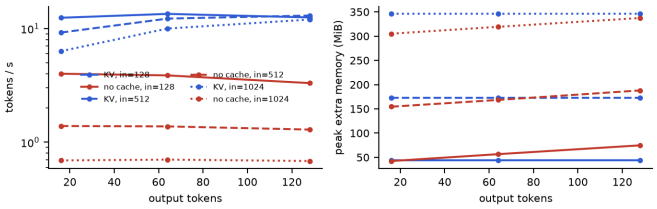


Fig. 6. Decode throughput (left) and peak extra memory (right) of the KV-cached decoder and the no-cache reference vs. input and output length.

Oversampling. Repeating the rare labels three times raised validation accuracy to 0.767 and macro-F1 to 0.621, because the model then predicted *excluded* correctly on both validation examples, while evidence F1 fell from 0.532 to 0.504. Its accuracy is about two standard deviations above the seed mean and its macro-F1 is only 0.013 above the best seed, while the run itself is a single sample, so it cannot separate this gain from chance. We therefore report it as promising but unproven, and we did not change the frozen model.

QLoRA. Training on a 4-bit NF4 base needed 3.4 GB instead of 4.3 GB (and ran at about the same speed per step in the memory benchmark), but accuracy was 0.709 (against 0.749 for bf16 LoRA, below every bf16 seed) and macro-F1 0.415, with an F1 of 0 on *not applicable*. The adapter was trained against the quantised weights and scored on the bf16 weights (the deployment setting here), so part of the drop can be this mismatch. On a 6 GB card both fit, so the extra memory saving did not justify the accuracy cost, which confirms the choice of bf16 LoRA.

Rank. Halving the LoRA rank to 8 trains 9.2 M instead of 18.5 M parameters and gives validation accuracy 0.733 and macro-F1 0.470, which is inside the seed-to-seed spread of the rank-16 runs (0.738–0.757 and 0.450–0.608). Within the noise we measured, rank 16 is therefore not shown to be better than rank 8. Doubling it to 32 (36.9 M trainable parameters) gave the highest validation accuracy of all runs, 0.772, but a macro-F1 of 0.469, because the *excluded* class (two examples) was missed. On the four classes with at least 12 validation examples (Table VI) rank 32 also has the best mean F1 (0.703 against 0.647 for the frozen run and 0.662–0.682 for the other rank-16 runs), driven by *included* (0.84) and *not enough information* (0.63). Its accuracy lies about two standard deviations above the three-seed mean, so the improvement is plausible but modest, about two accuracy points, and it comes from one run. It also needs more memory (4.55 against 4.28 GB peak) and its first attempt ran out of memory on the 6 GB card while other programs held GPU memory; it completed only after enabling expandable CUDA allocator segments. Lowering the learning rate to 10^{-4} gave accuracy 0.723 and macro-F1 0.462, below the baseline and with an F1 of 0 on *not applicable*: with the same three epochs the adapter is under-trained. Neither run changes the frozen model. The test split was evaluated once for the seed-42 rank-16 model and may not be re-scored to select a different configuration, so rank 32 is reported as a validated direction, not a result. The search remains small (one value of each hyper-parameter besides rank 16 and rate 2×10^{-4}, one training run each), so the frozen configuration is a reasonable default, not a tuned optimum.

Label calibration. The dominant validation error is *not enough information* predicted as *not excluded* (22 of 54 errors), so we tested a cheap post-hoc correction: score every allowed label by its sequence log-probability after the prompt prefix `{"label": "`, add a per-label bias, and fit the biases to maximise macro-F1 with leave-one-patient-out cross-validation over the eight validation patients (`evaluation/calibration.py`). Scoring by sequence probability alone gives accuracy 0.738 and macro-F1 0.411, below the greedy decoder's 0.543, because greedy decoding commits to the shared first token *not*. Calibration raises the cross-validated macro-F1 to 0.511 but lowers accuracy to 0.694, and it still does not exceed the greedy decoder. The biases chosen on all 206 examples (up to +3 nats) reach 0.599 macro-F1 only in-sample. We therefore did not adopt calibration; the gain is not reliable on eight patients with two to three examples of the rare classes.

F. KV-Cached Decoding

Table XII and Fig. 6 compare the two decoders. Prefill cost is the same for both, since both process the prompt once. Without a cache, every decode step reprocesses the whole sequence, so no-cache throughput falls from 4.0 to 0.7 tok/s as the input grows from 128 to 1024 tokens. With the cache, each step costs one token through the network and throughput stays between 9 and 13 tok/s. Decode speed-up therefore grows with context: about 3.6× at 128 input tokens, 10× at 512 and 18–20× at 1024.

Peak extra memory is dominated by prefill activations (the $T \times 151{,}936$ logit matrix), not by the cache. The cache adds 28 KiB per token, so it occupies only 32 MiB even at 1,152 tokens. At batch 1 on this GPU, decoding is launch-bound: a single-token step costs about 75–110 ms whatever the context length, which is why the cached throughput stays flat.

Equality. The CPU fp64 tests on tiny Qwen2 models and the bf16 GPU test on the base model produce token-identical output for both decoders. With the merged fine-tuned weights in bf16, **5 of the 9** benchmark cases are identical and 4 differ. The four differences have only two distinct causes: the three

TABLE VII
EVIDENCE-SENTENCE RETRIEVAL ON THE TEST SPLIT. MICRO POOLS TP/FP/FN; MACRO AVERAGES PER RECORD (BOTH-EMPTY = 1, SEE SEC. IV-A). NE = NON-EMPTY-GOLD SUBSET; ABST. = CORRECT EMPTY PREDICTIONS FOR EMPTY GOLD; UNS. = CITED IDS NOT IN THE NOTE.

Model	P_μ	R_μ	$F1_\mu$	P_M	R_M	$F1_M$	$F1_{NE}$	Abst.	Uns.
Base	0.097	0.649	0.169	0.374	0.899	0.354	0.374	53/145	0
Fine-tuned	0.500	0.638	0.561	0.888	0.904	0.819	0.663	131/145	0

TABLE VIII
RULE-ASSIGNED ERROR CATEGORIES, FINE-TUNED MODEL, TEST SPLIT (55 WRONG LABELS OF 206; ONE ERROR MAY HAVE SEVERAL).

Category	Count
correct label, wrong evidence	34
missing information treated as decisive (unsupported decision)	28
temporal qualifier	19
false 'not enough information' (evidence was present)	17
not-applicable confusion	13
negation in evidence	7
numeric / threshold reasoning	6
cited evidence where experts found none	3

cases with a 128-token input all diverge at step 5 on the same token pair, and the (1024 in, 128 out) case diverges at step 84. At both points the top-2 logit margin is 0.125, which is exactly one bf16 unit in the last place for logits between 16 and 32. In other words, the two candidate tokens are tied at bf16 resolution. The cached path computes attention for one query against stored keys, while the reference recomputes the full $T \times T$ product; the two reduction orders round differently, and a tie can go either way. Section V-I tests this explanation in fp32.

G. Tiled Attention

Against a genuine float64 ground truth (the reference routine run with float64 arithmetic and validated against an independent float64 softmax in the test suite), the tiled kernel matches PyTorch SDPA to the precision of its storage format (Table XIII). In fp32 its maximum error is 2.5×10^{-6}, against 2.8×10^{-6} for both our fp32 reference and SDPA, over 432 cases that cover GQA (12:2), decode shapes with S_q=1, causal and non-causal masks, and tile sizes 16, 64 and 128. In bf16 and fp16 all three implementations have the same error, set by rounding the output to the storage type. An earlier run of the same grid used an fp32 reference by mistake (the routine cast its inputs to fp32); it gave 2.7×10^{-6} for the tiled kernel and is kept, labelled as such, in `results/attention/fp32_reference_validation/`.

Speed and memory show the expected trade-off (Fig. 7). On the GTX 1660 SUPER (sm_75), PyTorch's fused flash and memory-efficient kernels are unavailable in bf16, so the fair baseline is the unfused `math` backend. At sequence length 4096, that backend materialises the score matrix and needs 1.9 GB of extra memory, whereas the tiled kernel needs 41 MiB, about 46× less. Being a Python loop over tiles, the tiled kernel is slower: 465 ms vs. 73 ms with 128-wide tiles. Larger tiles reduce the number of kernel launches and run 3.7× faster than 64-wide tiles. This is the I/O-versus-launch trade-off that fused kernels such as FlashAttention remove [18].

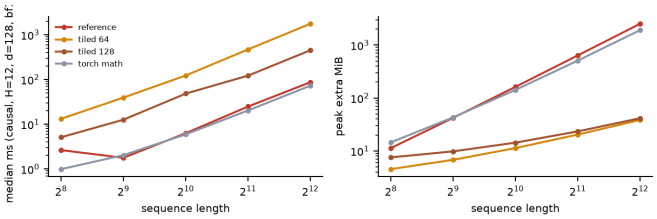


Fig. 7. Attention latency and peak extra memory vs. sequence length (bf16, GPU, 12 query / 2 KV heads, d=128, causal).

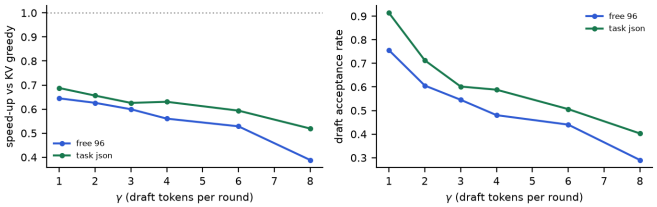


Fig. 8. Speculative decoding: acceptance rate and speed-up over KV-cached greedy decoding vs. draft length γ.

H. Speculative Decoding

The draft and target models share tokenizer, special tokens and chat template, which we verified programmatically. Acceptance is high: 91% of drafted tokens are accepted at γ=1 on the structured-output task, and acceptance falls to 40% at γ=8 (Table XIV, Fig. 8). The number of target forwards per output token falls from 1.0 to 0.31.

Even so, speculative decoding was *slower* than KV-cached greedy decoding at every γ, at 0.39–0.69× its speed. The reason is that at batch 1 on this GPU, forward cost scales with the number of layers and kernel launches rather than with parameter count. The 0.5B draft has 24 layers against the target's 28, so a draft step costs almost as much as a target step. We timed this directly, with greedy cached decoding after a 512-token prompt in bf16: one draft step took 189 ms and one target step took 239 ms, a ratio of 0.79, although the draft has a third of the target's parameters. Saving a target forward therefore saves less time than the extra draft steps add. Speculative decoding pays off when the draft is much cheaper per token than the target, for example a larger target, a shallower draft, or a compute-bound regime. We report the negative result as measured.

TABLE IX
VALIDATION-ONLY FINE-TUNING EXPERIMENTS (206 VALIDATION CRITERIA; THE TEST SPLIT IS UNTOUCHED). SAME PROMPT, PARSER AND GREEDY DECODER AS THE MAIN RUN; BEST EPOCH CHOSEN BY VALIDATION MACRO-F1; GB = PEAK GPU MEMORY. WALL-CLOCK TIME IS NOT REPORTED: THE GPU WAS SHARED WITH OTHER PROGRAMS.

Run	Acc	$MF1_p$	Evid F1	Incl acc	Excl acc	Best ep	GB
baseline (seed 42)	0.738	0.543	0.532	0.742	0.736	3	4.28
seed 43	0.757	0.608	0.579	0.697	0.786	3	4.28
seed 44	0.752	0.450	0.538	0.712	0.771	3	4.28
oversample rare labels 3×	0.767	0.621	0.504	0.712	0.793	3	4.28
QLoRA (4-bit NF4 base)	0.709	0.415	0.532	0.758	0.686	3	3.40
LoRA rank 8	0.733	0.470	0.513	0.727	0.736	3	4.15
LoRA rank 32	0.772	0.469	0.536	0.788	0.764	3	4.55
learning rate 10^{-4}	0.723	0.462	0.544	0.712	0.729	3	4.28

TABLE X
VARIATION ACROSS 3 TRAINING SEEDS (42, 43, 44), VALIDATION SPLIT. SD IS THE SAMPLE STANDARD DEVIATION.

Metric	Mean	SD	Per seed (42 / 43 / 44)
accuracy	0.749	0.010	0.738 / 0.757 / 0.752
macro_f1_present	0.534	0.079	0.543 / 0.608 / 0.450
evidence_micro_f1	0.550	0.025	0.532 / 0.579 / 0.538
inclusion_acc	0.717	0.023	0.742 / 0.697 / 0.712
exclusion_acc	0.764	0.026	0.736 / 0.786 / 0.771

A fine-tuned draft. The draft above is the untrained 0.5B model, so it is not adapted to the JSON task. We fine-tuned the 0.5B model with the same recipe (validation accuracy 0.748, macro-F1 0.445, Section V-E) and repeated the benchmark in the same session as an untrained-draft rerun (Table XV; the rerun reproduces the acceptance rates exactly and its greedy baseline is 10% slower than the earlier session, which is why the table uses the rerun). Acceptance at γ=4 rises from 0.59 to 0.85 and the speed-up from $0.62\times$ to $0.75\times$, with a best of $0.79\times$ at γ=6. Training the draft therefore removes most of the wasted drafting, but it is still *slower* than plain cached decoding, which confirms the diagnosis above: the limit is the draft's per-step cost, not its accuracy. On the task workload only the γ=8 row differs from greedy (as with the untrained draft, free-text generation differs more often because it has more steps at which a bf16 tie can occur; the mismatches are listed in `results/speculative/ftdraft/summary.json`).

I. Numerical Agreement in bf16 vs. fp32

All three mechanisms are exact in exact arithmetic, yet each produced a few bf16 outputs that differ from its reference. We checked whether these differences come from bugs or from rounding, using `speculative/investigate.py`, `attention/hf_integration.py` and `kvcache/fp32_check.py`.

Margins at the divergence point. Every bf16 divergence we found occurs where the two best logits differ by at most 0.125. For logits in $[16, 32)$, the spacing between adjacent bf16 values is $2^{4-7} = 0.125$, so the two tokens are tied, or one rounding step apart. For speculative example 0, the target logits at step 22 are 18.375 vs. 18.25 when tokens are processed one at a time, as in greedy decoding. Inside the 5-token verification block used by speculative decoding, the same two logits come out *exactly equal* (17.875 and 17.875), and the tie is broken the other way. The single-token and block paths differ by up to 0.84 in individual logits. Different matrix shapes select different GEMM kernels and reduction orders, and bf16 keeps only 8 bits of mantissa.

fp32 reruns. Table XVI repeats the diverging cases in fp32 on the CPU, without changing any code. Speculative decoding then matches greedy decoding token for token on all three investigated prompts. With the tiled kernel substituted for every attention layer of the fine-tuned model through the Transformers `AttentionInterface`, final-position logits stay within 3.4×10^{-4} of SDPA. Greedy output is identical on all three prompts, whereas in bf16 the logit gap is 0.5–0.7 and one prompt diverges. Finally, the KV-cached decoder matches the no-cache reference in fp32 on *all four* bf16-diverging KV cases: the three 128-token-input cases (16, 64 and 128 output tokens), which all diverged at step 5, and (1024 in, 128 out), which diverged at step 84. The fp32 runs took 59 min and 30 min on the CPU, mostly in the quadratic no-cache reference.

These checks are also part of the test suite. The real-model GPU test measures the logit gap at the first bf16 divergence and accepts it only if the gap is at most twice the measured difference between the two computation paths (observed gaps 0–0.125 against path differences 0.22–0.97; `results/speculative/tie_check.json`), and a control test confirms that a wrong token is rejected. A separate float32 test on the same real models, with no tolerance, passes for cached, no-cache and speculative decoding (3 prompts, 24 CPU minutes).

The mechanisms are therefore exact. In bf16 they are reproducible only up to ties within one rounding step, which is also true of production inference stacks. Any bf16 equality claim should be read with that caveat.

J. Application

Fig. 5 shows the running application, served on the same GPU used for evaluation. Each test used a note that is not in the dataset, together with criteria that we wrote ourselves:

- a 67-year-old man with type 2 diabetes and stage 3 chronic kidney disease;

TABLE XI

PER-CLASS F1 ON THE VALIDATION SPLIT AT THE FINAL EPOCH. THE LAST COLUMN AVERAGES THE FOUR LABELS WITH AT LEAST 12 VALIDATION EXAMPLES (*included*, *not excluded*, *not enough information*, *not applicable*); *excluded* (2 EXAMPLES) AND *not included* (3) ARE TOO SMALL TO RANK RUNS.

Run	incl.	not incl.	excl.	not excl.	NEI	N/A	Mean of 4
baseline (seed 42)	0.78	0.00	0.67	0.85	0.54	0.42	0.647
seed 43	0.74	0.00	1.00	0.88	0.55	0.48	0.662
seed 44	0.74	0.00	0.00	0.87	0.59	0.50	0.675
oversample rare labels $3\times$	0.77	0.00	1.00	0.88	0.60	0.48	0.682
QLoRA (4-bit NF4 base)	0.81	0.00	0.33	0.81	0.53	0.00	0.538
LoRA rank 8	0.76	0.00	0.67	0.85	0.54	0.00	0.537
LoRA rank 32	0.84	0.00	0.00	0.86	0.63	0.48	0.703
learning rate 10^{-4}	0.75	0.00	0.67	0.84	0.51	0.00	0.525

TABLE XII

KV-CACHED VS. NO-CACHE GREEDY DECODING (FINE-TUNED TARGET, BF16, BATCH 1, EOS DISABLED; MEAN OF 3 SYNCHRONISED RUNS AFTER 1 WARM-UP). PEAK = EXTRA MIB OVER RESIDENT WEIGHTS. EQ. = TOKEN-IDENTICAL OUTPUT.

In	Out	Pre ms	Dec ms	tok/s	Dec ms$_{nc}$	tok/s$_{nc}$	Speed-up	Peak	Peak$_{nc}$	Eq.
128	16	189	1098	12.4	3810	4.0	3.5	44	42	no@5
128	64	216	4546	13.4	16357	3.9	3.6	44	57	no@5
128	128	228	10014	12.5	38493	3.3	3.8	44	75	no@5
512	16	653	1084	9.2	10882	1.4	10.0	172	154	yes
512	64	667	4570	12.2	45849	1.4	10.0	172	168	yes
512	128	683	9163	13.0	98547	1.3	10.8	172	188	yes
1024	16	1380	1151	6.3	21827	0.7	19.0	346	305	yes
1024	64	1382	5026	10.0	89785	0.7	17.9	346	319	yes
1024	128	1352	9326	12.0	186221	0.7	20.0	346	337	no@84

TABLE XIII

ATTENTION AGREEMENT WITH A FLOAT64 REFERENCE OVER THE VALIDATION GRID (3 HEAD LAYOUTS INCL. GQA 12:2, 8 (S_q, S_k) SHAPES INCL. DECODE S_q=1, $d \in \{32, 64, 128\}$, CAUSAL ON/OFF; TILED ALSO OVER TILE SIZES 16/64/128).

Impl.	dtype	Cases	max $\lvert\Delta\rvert$	mean $\lvert\Delta\rvert$
reference	bfloat16	144	7.80e-03	1.83e-04
reference	float16	144	9.76e-04	2.28e-05
reference	float32	144	2.81e-06	2.46e-08
tiled	bfloat16	432	7.80e-03	1.83e-04
tiled	float16	432	9.76e-04	2.28e-05
tiled	float32	432	2.45e-06	2.04e-08
torch_sdpa	bfloat16	144	7.80e-03	1.83e-04
torch_sdpa	float16	144	9.76e-04	2.28e-05
torch_sdpa	float32	144	2.81e-06	2.53e-08

- a hypothetical add-on therapy trial with three inclusion and four exclusion criteria.

The note is split into numbered sentences, and each criterion is decoded and displayed as soon as it finishes. Each result card shows:

- the label, with both criterion types kept and all expert categories preserved;
- the cited sentences, which are also highlighted in the note;
- a rationale that passed the grounding check, or the deterministic template when it did not.

The overall outcome is shown together with the rule that produced it.

Invalid input is rejected with a field-level message and the user's draft is kept. This covers empty notes, malformed NCT IDs and unknown labels. An unreachable ClinicalTrials.gov service or a model that is still loading returns a clear message, not a stack trace.

The screenshots also exposed a model error. For the criterion "HbA1c between 7.0% and 10.5%", the model answered *not enough information*, although the note states an HbA1c of 8.4%, which is inside the range. In our first capture, the generated rationale also called 8.4% "outside the range" and contradicted the label, and it still passed the grounding check of Section III-F. We added two rules to the check:

- a rationale may not assert a definite verdict when the label is *not enough information*;
- a rationale may not use undecided wording when the label is definite.

Unit tests cover both rules. With them in place, the same case falls back to the evidence template (Fig. 5). The label error itself remains, and it is an example of the numeric-threshold errors in Table VIII.

The Inference Lab runs the no-cache, KV-cached and speculative decoders on the loaded model. For each run it reports timing, token equality and draft acceptance, and it also runs the attention check against a float64 ground truth. In the captured run (validation example 0, γ=4; Fig. 5, bottom left), the output is a 15-token JSON answer and the draft proposed every token correctly (100% acceptance). Speculative decoding then took 2,769 ms against 3,533 ms for KV-cached greedy decoding, a speed-up of $1.28\times$, and all three decoders produced identical tokens. Live timings on a shared consumer GPU vary from run to run, so one run is

TABLE XIV
GREEDY SPECULATIVE DECODING (TARGET: FINE-TUNED QWEN2.5-1.5B, DRAFT: QWEN2.5-0.5B, BF16) VS. KV-CACHED GREEDY ON THE SAME 8 VALIDATION PROMPTS. ACC. = ACCEPTED/PROPOSED; $\bar{n}$ = ACCEPTED PER ROUND; FWD/TOK = TARGET FORWARDS PER OUTPUT TOKEN.

Workload	γ	Lat. ms	tok/s	Acc.	$\bar{n}$	Fwd/tok	Speed-up	Eq.
task_json	greedy	1437	9.9	–	–	1.00	1.00	yes
task_json	1	2089	6.8	0.91	0.91	0.58	0.69	yes
task_json	2	2190	6.5	0.71	1.43	0.48	0.66	yes
task_json	3	2296	6.3	0.60	1.80	0.43	0.63	no
task_json	4	2279	6.3	0.59	2.35	0.37	0.63	yes
task_json	6	2420	5.9	0.51	3.04	0.32	0.59	yes
task_json	8	2767	5.1	0.40	3.22	0.31	0.52	yes
free_96	greedy	7345	13.1	–	–	1.00	1.00	yes
free_96	1	11392	8.4	0.76	0.76	0.58	0.64	no
free_96	2	11730	8.2	0.61	1.21	0.46	0.63	no
free_96	3	12251	7.8	0.55	1.62	0.39	0.60	no
free_96	4	13105	7.3	0.48	1.90	0.35	0.56	no
free_96	6	13899	6.9	0.44	2.59	0.29	0.53	no
free_96	8	18911	5.1	0.29	2.25	0.32	0.39	no

TABLE XV
SPECULATIVE DECODING ON THE REAL TASK (JSON ANSWERS) WITH THE UNTRAINED VS. THE FINE-TUNED 0.5B DRAFT; TARGET = FINE-TUNED 1.5B, SAME 8 VALIDATION PROMPTS, BF16, BATCH 1. SPEED-UP IS RELATIVE TO KV-CACHED GREEDY DECODING OF THE TARGET.

γ	Acc. (untrained)	Speed-up	Acc. (fine-tuned)	Speed-up
1	0.91	0.67	0.96	0.69
2	0.71	0.64	0.91	0.73
4	0.59	0.62	0.85	0.75
6	0.51	0.59	0.88	0.79
8	0.40	0.51	0.59	0.64

TABLE XVI
TOKEN AGREEMENT WITH THE REFERENCE IN BF16 (GPU) VS. FP32 (CPU), ON THE CASES THAT DIVERGED IN BF16.

Check	bf16 GPU	fp32 CPU
Speculative vs. greedy (γ=4, 3 prompts)	1/3 equal	3/3 equal
Tiled attention in model vs. SDPA (3 prompts)	2/3 equal	3/3 equal
max $\lvert\Delta\text{logit}\rvert$	0.70	3.4×10^{-4}
KV-cached vs. no-cache (4 diverging cases)	0/4 equal	4/4 equal

not a stable figure. We therefore repeated the identical request five times (`results/lab/live_runs.json`): the speed-up was $1.43 \pm 0.22\times$ (range 1.18–1.73×), and the four runs saved earlier in the application database gave 1.28–1.57× (`results/lab/ui_runs_saved.json`); the screenshot shows one of them. In every run the tokens were identical and acceptance was 100%. Averaged over the 8 benchmark prompts at γ=4, acceptance was 59% and the result was a slowdown (Table XIV). Whether speculative decoding helps depends almost entirely on acceptance: each fully accepted round produces $\gamma+1$ tokens for one target forward pass.

VI. LIMITATIONS

Data scale and label skew. The benchmark has 53 patients; the prescribed split leaves 603 training annotations and only 8 test patients (206 annotations). Rare labels are nearly absent: the test split has no *excluded* example and only 7 *not included*, so per-class scores for these classes are either undefined or rest on a handful of cases, and patient-level bootstrap intervals are wide.

Seeds and search. The frozen model is a single run (seed 42). Two further seeds, oversampling, QLoRA, rank-8 and rank-32 runs and a lower learning rate (Section V-E, validation split only) show that validation accuracy varies by about one point between seeds but macro-F1 by up to 0.16, because the rare classes have 2–3 validation examples. The hyper-parameter search is small (one training run per variant), so no configuration is shown to beat the frozen one beyond noise; rank 32 has the highest validation accuracy but is a single run, and it could not be used for the test split, which was evaluated once. Test-set numbers come only from the frozen seed-42 model.

Explanations. Rationales are constrained to the cited sentences and pass an automatic grounding check, but they were not rated by clinicians, and the check (numbers and label contradiction) cannot catch every unfaithful paraphrase. They should be read as summaries of the cited evidence, not as clinical reasoning.

Inference measurements. Timings are from one consumer GPU under Windows with a slow host CPU; at batch size 1 small kernels are launch-bound, which shapes every speed result (Section V). The tiled attention kernel is an educational PyTorch implementation, not a fused kernel. It was checked inside the full model (Section V-I) but is not used in the application's serving path. Equality claims in bf16 hold only up to logit ties within one rounding step; exactness was confirmed in fp32 on all four diverging KV-cache cases and on three prompts for speculative decoding, not on every prompt of the benchmark grid.

Aggregation and scope. The overall outcome is a hand-written rule over criterion predictions; the dataset contains no patient–trial ground truth, so this outcome is not evaluated. Criteria fetched from ClinicalTrials.gov are split heuristically. ELIGITRACE is an educational screening prototype and must not be used to make enrollment or treatment decisions.

VII. Conclusion

ELIGITRACE shows that a 1.5B-parameter open model, adapted with LoRA on a single 6 GB consumer GPU in about an hour, reaches 0.733 accuracy on held-out patients and cites the same evidence sentences as expert annotators far more precisely than its untuned starting point. Every component runs locally, so patient notes never leave the machine.

Each inference mechanism has its own lesson:

- *KV cache*: removes the quadratic cost of recomputation, and its benefit grows with context length;
- *online softmax*: removes the quadratic memory of attention, but needs kernel fusion before it also saves time;
- *speculative decoding*: is exact, but at batch 1 on launch-bound hardware it only pays off when the draft is much cheaper per step than the target, not merely smaller.

In reduced precision, agreement with the reference is "identical up to ties within one rounding step", and a careful comparison has to say so explicitly.

The weakest areas are the rare labels and reasoning from the absence of a fact. Natural next steps are more annotated data for *not included* and *not applicable*, label-balanced or contrastive training, clinician review of the generated explanations, and a larger validation study: our three-seed runs show that validation macro-F1 varies by about 0.16 between seeds, so single-run differences in the rare classes cannot be trusted. Oversampling the rare labels, a fine-tuned draft model and a lower LoRA rank were all tested on the validation split (Section V-E); none is shown to help beyond noise except that a fine-tuned draft raises speculative acceptance from 0.59 to 0.85 (still slower than plain decoding).

References

[1] Q. Jin, Z. Wang, C. S. Floudas, F. Chen, C. Gong, D. Bracken-Clarke, E. Xue, Y. Yang, J. Sun, and Z. Lu, "Matching patients to clinical trials with large language models," *Nature Communications*, vol. 15, p. 9074, 2024.

[2] M. Wornow, A. Lozano, D. Dash, J. Jindal, K. W. Mahaffey, and N. H. Shah, "Zero-shot clinical trial patient matching with LLMs," *arXiv preprint arXiv:2402.05125*, 2024.

[3] Qwen Team, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.

[4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Int. Conf. Learning Representations (ICLR)*, 2022.

[5] B. Koopman and G. Zuccon, "A test collection for matching patients to clinical trials," in *Proc. 39th Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2016, pp. 669–672.

[6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[7] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, p. 127063, 2024.

[8] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 4895–4901.

[9] A. Grattafiori *et al.*, "The Llama 3 herd of models," 2024, arXiv preprint arXiv:2407.21783.

[10] Gemma Team, "Gemma 2: Improving open language models at a practical size," 2024, arXiv preprint arXiv:2408.00118.

[11] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[12] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, "Mixed precision training," in *Int. Conf. Learning Representations (ICLR)*, 2018.

[13] T. Chen, B. Xu, C. Zhang, and C. Guestrin, "Training deep nets with sublinear memory cost," *arXiv preprint arXiv:1604.06174*, 2016.

[14] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently scaling transformer inference," in *Proc. Machine Learning and Systems (MLSys)*, vol. 5, 2023.

[15] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th Symp. Operating Systems Principles (SOSP)*, 2023, pp. 611–626.

[16] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," *arXiv preprint arXiv:1805.02867*, 2018.

[17] M. N. Rabe and C. Staats, "Self-attention does not need $O(n^2)$ memory," *arXiv preprint arXiv:2112.05682*, 2021.

[18] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[19] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *Int. Conf. Learning Representations (ICLR)*, 2024.

[20] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proc. 40th Int. Conf. Machine Learning (ICML)*, 2023, pp. 19 274–19 286.

[21] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," *arXiv preprint arXiv:2302.01318*, 2023.

[22] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Int. Conf. Learning Representations (ICLR)*, 2019.

Appendix A
AI-Use Statement

Claude (Anthropic) was used as an AI assistant to help with literature search, planning, code scaffolding, debugging and drafting text. All code was reviewed, executed and tested by the author (the automated test suite covers the data split, leakage, parser, metrics, attention, KV-cache and speculative-decoding equality, and the API), all reported numbers come from runs in this repository, and the author can explain, execute and modify every component. No AI-generated content was used as data, labels or evaluation signal.

Appendix B
Reproducibility

Source code, configuration, split manifest, logs and results: https://github.com/ruhank-dev/eligitrace. LoRA adapter: https://huggingface.co/arceus1rk/eligitrace-qwen2.5-1.5b-lora. With `uv` and `Task` installed:

```
task setup     # env from uv.lock
task data      # split, manifest, stats
task train     # LoRA fine-tuning
task eval-final  # one-time test eval
task bench-kv; task bench-attn
task bench-spec
task download-weights
task app       # 127.0.0.1:8000
```